Joomla! Security

Edisi Ubuntu Server 26.04 LTS

Panduan Keselamatan CMS Joomla!
untuk Apache, PHP-FPM, MySQL,
UFW, ModSecurity CRS dan Let's Encrypt



```
$ sudo ufw status
Status: active

$ sudo systemctl status apache2
Active: active (running)

$ sudo certbot --nginx
Successfully received certificate.
Certificate is saved at:
/etc/letsencrypt/live/your-domain/
```

APACHE | PHP-FPM | MYSQL | UFW | MODSECURITY CRS | LET'S ENCRYPT

MANUAL TEKNIKAL

SELAMAT • TERKAWAL • DIPERCAYAI

Joomla! Security
Manual Teknikal Pengukuhan

Panduan keselamatan CMS Joomla! untuk Apache, PHP-FPM, MySQL, UFW, ModSecurity CRS dan Let’s Encrypt pada Ubuntu Server 26.04 LTS.

Shaiffulnizam bin Mohamad
security.apasaja.my

Edisi 2026 · Bahasa Melayu · Kemas kini *Joomla Security 2015*
Selamat · Terkawal · Dipercayai

Joomla! Security — Manual Teknikal Pengukuhan

Edisi Ubuntu Server 26.04 LTS · Edisi Pertama, 2026

Hak cipta

© 2026. Hak cipta terpelihara. Manual ini boleh digunakan dan diedarkan untuk tujuan pembelajaran dan pentadbiran dalaman. Pengeluaran semula untuk jualan komersial memerlukan kebenaran bertulis pemilik hak cipta. Nama produk seperti Joomla!, Ubuntu, Apache, MySQL, Cloudflare dan Let's Encrypt ialah tanda dagangan pemilik masing-masing dan digunakan di sini secara rujukan sahaja.

Penafian

Manual ini disediakan 'sebagaimana adanya' tanpa sebarang jaminan. Keselamatan siber ialah bidang yang sentiasa berubah; arahan yang tepat pada hari ini mungkin memerlukan pelarasan pada masa depan. Penulis dan penyumbang tidak bertanggungjawab atas sebarang kerugian, kerosakan data, atau gangguan perkhidmatan akibat penggunaan panduan ini. Sentiasa uji pada persekitaran pementasan dan simpan sandaran yang telah disahkan sebelum membuat perubahan pada sistem produksi.

Tentang manual ini

Manual ini ialah kemas kini kepada buku *Joomla Security 2015* yang asalnya ditulis oleh penulis sebagai bahan kursus keselamatan asas untuk web CMS Joomla. Sejak 2015, landskap keselamatan telah banyak berubah — Ubuntu, PHP, dan Joomla telah beberapa versi ke hadapan, TLS percuma menjadi kebiasaan melalui Let's Encrypt, WAF seperti OWASP CRS 4 semakin matang, dan CDN seperti Cloudflare kini mudah diakses. Edisi ini mengemas kini setiap arahan kepada Ubuntu Server 26.04 LTS dan amalan terbaik semasa, serta menambah bab baharu seperti pemulihan selepas diserang, CDN, dan integrasi alat imbasan SecScan.

Kandungannya berasaskan dokumentasi rasmi Joomla, OWASP, serta amalan terbaik industri, dan dipetakan kepada semakan automatik yang dilakukan oleh SecScan di `https://security.apasaja.my`. Ia melengkapi — bukan menggantikan — pertimbangan pakar keselamatan untuk persekitaran berisiko tinggi.

Tentang penulis

Shaiffulnizam bin Mohamad ialah pengamal keselamatan web dan pentadbir sistem yang telah lama terlibat dalam mengukuhkan dan menyelenggara laman berasaskan CMS. Beliau menulis edisi pertama *Joomla Security* pada 2015 sebagai bahan kursus keselamatan asas, dan meneruskan kerja itu melalui SecScan — sebuah alat imbasan keselamatan web yang membantu pemilik laman mengenal pasti dan membetulkan kelemahan konfigurasi.

Tentang security.apasaja.my

`https://security.apasaja.my` ialah platform SecScan — alat imbasan keselamatan web percuma yang menyemak laman anda secara luaran (TLS, header, DNS, fail terdedah, dan CVE CMS), memberi gred keseluruhan, dan menyenaraikan cadangan pembetulan yang boleh ditindaki. Cara menggunakannya dan mentafsir keputusan diterangkan dalam Bab 18.

Maklum balas dan pembetulan dialu-alukan supaya edisi akan datang menjadi lebih baik. Lawati security.apasaja.my untuk versi terkini.

Kandungan

Pengenalan

Falsafah pertahanan berlapis dan seni bina pelayan

Bab 1 · Persediaan pelayan

Kemas kini, pengguna sudo, zon masa, hardening awal

Bab 2 · Firewall UFW

Dasar default-deny, port perlu, kadar sambungan

Bab 3 · Pengukuhan SSH

Kunci awam, matikan root & kata laluan, port

Bab 4 · Apache

Header keselamatan (HSTS, CSP), sembunyi fingerprint, sekat fail

Bab 5 · PHP-FPM

Kolam khusus, disable_functions, open_basedir, had

Bab 6 · MySQL

Pemasangan selamat, pengguna & keistimewaan minimum

Bab 7 · Let's Encrypt (TLS)

Certbot, HSTS, auto-renew, konfigurasi SSL kukuh

Bab 8 · ModSecurity + OWASP CRS

WAF, CRS 4, paranoia level, pengecualian Joomla

Bab 9 · Pengukuhan Joomla

Pemasangan, configuration.php, .htaccess, akaun

Bab 10 · Fail2ban

Jail SSH, Apache auth, Joomla admin login

Bab 11 · Backup & pemulihan

Fail + pangkalan data, automasi, ujian pulih

Bab 12 · Pemantauan & penyelenggaraan

Log, integriti fail, kemas kini berkala

Bab 13 · CDN & Cloudflare

Pertahanan pinggir: WAF, DDoS, sembunyi IP asal

Bab 14 · Pemulihan selepas diserang

Kuarantin, forensik, pembersihan, pemulihan

Bab 15 · Nginx (alternatif Apache)

Blok pelayan Joomla, try_files, sekat fail, TLS

Bab 16 · Jailkit dengan Virtualmin

Kurung pengguna SSH dalam chroot jail

Bab 17 · .htaccess & robots.txt

Peraturan pengukuhan .htaccess dan salah faham robots.txt

Bab 18 · Menggunakan SecScan

Menjalankan scan, memahami gred/skor, mentafsir keputusan

Bab 19 · Senarai semak akhir

Audit ringkas sebelum ke produksi

Pengenalan

Joomla! ialah sistem pengurusan kandungan (CMS) yang matang dan fleksibel, tetapi seperti mana-mana aplikasi web yang terdedah ke internet, ia menjadi sasaran serangan automatik yang berterusan. Menurut dokumentasi keselamatan rasmi Joomla, majoriti pencerobohan bukan berpunca daripada teras Joomla, sebaliknya daripada **sambungan pihak ketiga**, **konfigurasi pelayan yang lemah**, dan **amalan pentadbiran yang cuai**. Manual ini menangani ketiga-tiganya.

Prinsip yang mendasari keseluruhan buku ini ialah **pertahanan berlapis** (defense in depth): tiada satu lapisan pun yang sempurna, jadi kita menyusun beberapa lapisan supaya kegagalan satu lapisan tidak bermakna keseluruhan sistem tumbang. Firewall menyekat akses, SSH dikunci dengan kunci awam, Apache dan PHP-FPM dikonfigur seketat mungkin, MySQL diberi keistimewaan minimum, ModSecurity menapis muatan berniat jahat, dan Fail2ban menyekat penyerang berulang.

Seni bina pelayan

Rajah di bawah menunjukkan bagaimana setiap lapisan disusun. Trafik pelawat melalui setiap lapisan dari atas ke bawah — setiap satu menapis atau mengehadkan apa yang boleh lalu ke lapisan seterusnya. Lapisan silang (Fail2ban, sandaran, pemeriksaan integriti) beroperasi merentas keseluruhan sistem.



Rajah 1 — Seni bina pertahanan berlapis untuk pelayan Joomla di Ubuntu Server 26.04 LTS.

Untuk siapa manual ini

Manual ini ditujukan kepada pentadbir sistem, pembangun web, dan pemilik laman yang menguruskan Joomla di atas pelayan Ubuntu mereka sendiri (VPS atau dedicated). Anda dijangka biasa dengan baris arahan Linux asas. Setiap arahan disertakan penuh supaya boleh disalin terus.

Andaian persekitaran

Sistem operasi	Ubuntu Server 26.04 LTS (64-bit)
Pelayan web	Apache 2.4 dengan MPM event + PHP-FPM
PHP	PHP 8.5 (lalai 26.04) atau 8.4 melalui PPA Ondrej Sury
Pangkalan data	MySQL 8.x (atau MariaDB setara)
CMS	Joomla 5.x LTS
Akses	Pengguna sudo bukan-root; akses SSH

AMARAN — Uji sebelum produksi

Setiap konfigurasi dalam manual ini boleh menjejaskan ketersediaan laman jika disalah tetapkan. Uji pada pelayan pementasan (staging) dahulu, dan sentiasa simpan sandaran yang telah diuji sebelum membuat perubahan besar. Sesuaikan nombor versi PHP (contoh **8.5** atau **8.4**) mengikut yang benar-benar dipasang pada pelayan anda.

BAB 1

Persediaan pelayan

Asas yang selamat sebelum sebarang perkhidmatan dipasang

Sebelum memasang Apache atau Joomla, kita kukuhkan asas sistem terlebih dahulu. Langkah ini mengurangkan permukaan serangan sejak awal.

1.1 Kemas kini sistem

Pastikan semua pakej terkini. Versi lama mengandungi kelemahan yang maklumat eksploitnya sudah tersebar.

```
$ sudo apt update && sudo apt upgrade -y
$ sudo apt autoremove --purge -y
```

Aktifkan kemas kini keselamatan automatik supaya tampung dipasang tanpa campur tangan manual:

```
$ sudo apt install -y unattended-upgrades
$ sudo dpkg-reconfigure --priority=low unattended-upgrades
```

1.2 Cipta pengguna sudo bukan-root

Jangan sekali-kali beroperasi sebagai root untuk kerja harian. Cipta pengguna biasa dengan hak sudo:

```
$ sudo adduser deploy
$ sudo usermod -aG sudo deploy
# log masuk semula sebagai 'deploy'
```

> **NOTA — Zon masa & NTP**
> Tetapkan zon masa yang betul supaya cap masa log tepat semasa forensik: `sudo timedatectl set-timezone Asia/Kuala_Lumpur`. Penyegerakan masa (NTP) lalai aktif melalui systemd-timesyncd.

1.3 Hardening kernel ringkas (sysctl)

Beberapa tetapan rangkaian melindungi daripada spoofing dan serangan lapisan IP. Cipta fail konfigurasi:

```
$ sudo nano /etc/sysctl.d/99-hardening.conf
```

```
# Lindungi daripada IP spoofing
net.ipv4.conf.all.rp_filter = 1
net.ipv4.conf.default.rp_filter = 1
# Abai ICMP redirect & source routing
net.ipv4.conf.all.accept_redirects = 0
net.ipv4.conf.all.accept_source_route = 0
# Log paket suspicious (martian)
net.ipv4.conf.all.log_martians = 1
# Lindung SYN flood
net.ipv4.tcp_syncookies = 1
```

```
$ sudo sysctl --system
```

1.4 Ringkasan bab

- Sistem dikemas kini dan kemas kini automatik diaktifkan.
- Pengguna sudo bukan-root digunakan untuk semua kerja.
- Zon masa betul dan tetapan sysctl pengukuhan dipasang.

BAB 2

Firewall dengan UFW

Hanya benarkan apa yang perlu; sekat yang lain

UFW (Uncomplicated Firewall) ialah antara muka mesra untuk iptables/nftables. Strategi kita mudah: **tolak semua masuk secara lalai**, kemudian benarkan hanya port yang benar-benar diperlukan.

2.1 Dasar asas

```
$ sudo apt install -y ufw
$ sudo ufw default deny incoming
$ sudo ufw default allow outgoing
```

AMARAN — Benarkan SSH dahulu
Sebelum `ufw enable`, WAJIB benarkan port SSH anda — jika tidak anda akan terkunci daripada pelayan. Jika anda menukar port SSH (Bab 3), benarkan port baharu itu, bukan 22.

2.2 Benarkan perkhidmatan perlu

```
# SSH (tukar 22 jika port anda berbeza)
$ sudo ufw allow 22/tcp
# HTTP & HTTPS untuk Apache
$ sudo ufw allow 80/tcp
$ sudo ufw allow 443/tcp
```

Hadkan kadar cubaan sambungan SSH untuk melembapkan serangan brute-force pada peringkat firewall:

```
$ sudo ufw limit 22/tcp
```

2.3 Aktifkan & sahkan

```
$ sudo ufw enable
$ sudo ufw status verbose
```

Output sepatutnya menunjukkan dasar `deny (incoming)` dengan hanya port 22, 80 dan 443 dibenarkan. Semak semula bahawa tiada port pangkalan data (3306) terdedah ke internet.

NOTA — MySQL jangan didedah
MySQL sepatutnya hanya mendengar pada `127.0.0.1` (Bab 6). Jangan sekali-kali `ufw allow 3306` melainkan anda benar-benar memerlukan akses jauh dan telah menghadkannya kepada IP tertentu.

BAB 3

Pengukuhan SSH

Kunci awam, bukan kata laluan

SSH ialah pintu masuk pentadbiran pelayan anda. Ia juga sasaran brute-force paling kerap. Matlamat bab ini: hanya membenarkan log masuk melalui kunci awam, mematikan log masuk root, dan mengurangkan bunyi serangan.

3.1 Sediakan kunci SSH

Pada komputer **tempatan** anda (bukan pelayan), jana pasangan kunci jika belum ada, kemudian salin kunci awam ke pelayan:

```
# di komputer tempatan
$ ssh-keygen -t ed25519 -C "admin@laman"
$ ssh-copy-id deploy@ALAMAT_IP_PELAYAN
```

3.2 Kukuhkan sshd_config

Sunting konfigurasi daemon SSH. Amalan terbaik semasa ialah meletakkan tetapan dalam fail berasingan di bawah `/etc/ssh/sshd_config.d/` supaya tidak ditimpa semasa naik taraf:

```
$ sudo nano /etc/ssh/sshd_config.d/99-hardening.conf
```

```
# Matikan log masuk root sepenuhnya
PermitRootLogin no
# Hanya kunci awam — matikan kata laluan
PasswordAuthentication no
KbdInteractiveAuthentication no
PubkeyAuthentication yes
# Hadkan pengguna yang boleh log masuk
AllowUsers deploy
# Kurangkan bunyi & tetingkap serangan
MaxAuthTries 3
LoginGraceTime 20
X11Forwarding no
```

AMARAN — Uji sebelum tutup sesi

Selepas `sudo systemctl restart ssh`, JANGAN tutup sesi semasa. Buka terminal **baharu** dan pastikan anda boleh log masuk dengan kunci. Jika gagal, anda masih ada sesi lama untuk membaiki tetapan.

```
$ sudo systemctl restart ssh
```

3.3 Menukar port SSH (pilihan)

Menukar port bukan langkah keselamatan sebenar, tetapi mengurangkan bunyi log daripada bot automatik. Jika anda berbuat demikian, kemas kini UFW terlebih dahulu:

```
# dalam 99-hardening.conf
Port 2222
```

```
$ sudo ufw allow 2222/tcp
$ sudo ufw delete allow 22/tcp
$ sudo systemctl restart ssh
```

BAB 4

Apache

Pelayan web yang kukuh dan senyap

Apache menghidangkan laman Joomla anda. Konfigurasi lalai agak longgar untuk keselamatan produksi — bab ini mengetatkannya: mematikan pendedahan maklumat, menambah header keselamatan, dan menyekat akses ke fail sensitif Joomla.

4.1 Pemasangan

```
$ sudo apt install -y apache2
# modul yang kita perlukan
$ sudo a2enmod headers rewrite ssl
$ sudo systemctl enable --now apache2
```

4.2 Matikan pendedahan maklumat

Secara lalai Apache mendedahkan versi dan sistem operasinya dalam header dan halaman ralat. Sunting fail keselamatan global:

```
$ sudo nano /etc/apache2/conf-available/security.conf
```

```
# Sembunyikan versi Apache & OS
ServerTokens Prod
ServerSignature Off
# Halang laman disematkan (clickjacking)
TraceEnable Off
```

4.3 Header keselamatan — kenapa ia penting

Header keselamatan HTTP ialah arahan yang pelayan hantar kepada pelayar untuk memberitahunya **cara berkelakuan dengan selamat** semasa memaparkan laman anda. Ia percuma, tidak menjejaskan prestasi, dan menutup keseluruhan kelas serangan — clickjacking, penurunan taraf HTTPS, MIME-sniffing, dan banyak XSS. Yang penting: penguatkuasaan berlaku di **dalam pelayar pelawat**, jadi ia melindungi walaupun terhadap kelemahan yang belum anda kesan. Inilah antara ‘quick win’ keselamatan paling berbaloi.

Berikut ialah header utama, apa yang dilakukannya, dan sebab ia perlu:

Strict-Transport-Security (HSTS)	Memaksa pelayar sentiasa guna HTTPS untuk domain anda, walaupun pengguna menaip http://. Menghalang serangan penurunan taraf (SSL stripping) dan pemintasan. Hanya ditambah selepas TLS berfungsi (Bab 7).

Content-Security-Policy (CSP)	Mengawal sumber (skrip, gaya, imej) yang pelayar dibenarkan memuatkan. Ini pertahanan paling kuat terhadap XSS — walaupun penyerang menyuntik skrip, pelayar enggan melaksanakannya jika bukan dari sumber yang dibenarkan.
X-Content-Type-Options	Nilai 'nosniff' menghalang pelayar meneka jenis fail. Menghalang serangan yang menyamarkan skrip sebagai imej atau teks.
X-Frame-Options	'SAMEORIGIN' menghalang laman anda disematkan (iframe) di laman lain — pertahanan terhadap clickjacking, di mana pengguna ditipu mengklik elemen tersembunyi.
Referrer-Policy	Mengawal berapa banyak maklumat URL dihantar apabila pengguna klik keluar. Menghalang kebocoran laluan sensitif atau token dalam URL.
Permissions-Policy	Mematikan ciri pelayar yang tidak diperlukan laman anda (kamera, mikrofon, geolokasi), mengurangkan permukaan serangan.

Tambah header ini dalam VirtualHost laman anda (atau fail conf global). Modul `headers` mesti aktif (Bab 4.1):

```
Header always set X-Content-Type-Options "nosniff"
Header always set X-Frame-Options "SAMEORIGIN"
Header always set Referrer-Policy "strict-origin-when-cross-origin"
Header always set Permissions-Policy "geolocation=(), camera=(), microphone=()"
# HSTS — hanya selepas HTTPS berfungsi (Bab 7)
Header always set Strict-Transport-Security "max-age=63072000; includeSubDomains"
```

4.4 Content-Security-Policy dengan berhati-hati

CSP ialah header paling berkuasa tetapi paling mudah merosakkan laman jika terlalu ketat — Joomla dan sambungannya kerap memuatkan skrip sebaris (inline) dan aset luaran. Oleh itu, mulakan dalam mod **Report-Only** yang melaporkan pelanggaran tanpa menyekat apa-apa, pantau, kemudian ketatkan.

```
# Fasa 1: pantau sahaja, tidak menyekat
Header always set Content-Security-Policy-Report-Only \
  "default-src 'self'; img-src 'self' data:; report-uri /csp-report"
```

Setelah anda yakin tiada aset sah yang tersekat, tukar kepada penguatkuasaan sebenar. Dasar asas yang munasabah untuk kebanyakan laman Joomla:

```
Header always set Content-Security-Policy \
  "default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'; \
   img-src 'self' data:; frame-ancestors 'none'; base-uri 'self'; \
   form-action 'self'; upgrade-insecure-requests"
```

AMARAN — CSP boleh merosakkan laman

Dasar CSP yang salah boleh menghalang JavaScript, imej, atau borang berfungsi. SENTIASA uji dalam mod Report-Only dahulu, dan uji setiap templat/sambungan. `frame-ancestors 'none'` memberikan perlindungan clickjacking yang lebih moden berbanding X-Frame-Options.

Muat semula Apache selepas setiap perubahan, dan sentiasa uji sintaks dahulu:

```
$ sudo apache2ctl configtest
$ sudo systemctl reload apache2
```

4.5 VirtualHost untuk Joomla

Konfigurasi tapak dengan direktori root yang betul dan blok akses selamat. Andaikan laman di `/var/www/joomla`:

```
$ sudo nano /etc/apache2/sites-available/joomla.conf
```

```
<VirtualHost *:80>
    ServerName contoh.my
    DocumentRoot /var/www/joomla

    <Directory /var/www/joomla>
        Options -Indexes +FollowSymLinks
        AllowOverride All
        Require all granted
    </Directory>

    ErrorLog ${APACHE_LOG_DIR}/joomla_error.log
    CustomLog ${APACHE_LOG_DIR}/joomla_access.log combined
</VirtualHost>
```

`Options -Indexes` penting: ia menghalang penyenaraian direktori jika fail indeks tiada. `AllowOverride All` membenarkan fail .htaccess Joomla berfungsi.

4.6 Sekat fail sensitif Joomla

Fail seperti `configuration.php` dan direktori pentadbiran tidak sepatutnya diakses terus. Tambah dalam blok Directory:

```
# Halang akses terus ke configuration.php
<Files "configuration.php">
    Require all denied
</Files>
# Sekat fail tersembunyi & backup
<FilesMatch "(^\.|\.(bak|save|old|sql|log)$)">
    Require all denied
</FilesMatch>
```

```
$ sudo a2ensite joomla
$ sudo a2dissite 000-default
$ sudo systemctl reload apache2
```

4.7 Sembunyikan ciri Joomla & jenis server

Penyerang bermula dengan **fingerprinting** — mengenal pasti perisian dan versi yang anda jalankan supaya mereka boleh mencari eksploit yang sepadan. Menyembunyikan butiran ini tidak menggantikan pengukuhan sebenar, tetapi ia melambatkan penyerang automatik dan menyukarkan penyasaran. Ini dipanggil 'security through obscurity' — berguna sebagai lapisan tambahan, bukan pertahanan tunggal.

1. Sembunyikan versi & jenis pelayan. Kita telah tetapkan `ServerTokens Prod` (Bab 4.2), yang menukar header `Server: Apache/2.4.x (Ubuntu)` kepada sekadar `Server: Apache`. Untuk menyembunyikan sepenuhnya, gunakan modul `mod_security2` (Bab 8):

```
# dalam konfigurasi ModSecurity
SecServerSignature " "
```

2. Buang X-Powered-By (versi PHP). PHP mendedahkan versinya dalam header. Matikan dalam php.ini (Bab 5): `expose_php = Off`. Ini membuang header `X-Powered-By: PHP/8.x`.

3. Buang tag generator Joomla. Secara lalai Joomla menambah `<meta name="generator" content="Joomla! ...">` dalam setiap halaman, yang mengumumkan bahawa laman itu Joomla. Buang melalui override templat atau plugin sistem ringkas:

```
// Plugin sistem – pada event onBeforeCompileHead
$doc = Factory::getApplication()->getDocument();
$doc->setGenerator('');
```

Sebagai alternatif tanpa kod, banyak sambungan keselamatan Joomla (contoh keluarga 'Admin Tools') menawarkan pilihan 'Hide/customise generator meta tag' dengan satu klik.

4. Alih keluar fail yang mendedahkan versi. Joomla menyertakan fail yang menyatakan nombor versi tepat — berguna untuk penyerang memadankan CVE. Hadkan aksesnya melalui Apache atau buang jika tidak perlu:

```
# Sekat fail manifes/versi yang mendedahkan maklumat
<FilesMatch "(README\.txt|CHANGELOG\.php|joomla\.xml|\.md)$">
    Require all denied
</FilesMatch>
```

5. Ubah suai halaman ralat. Halaman ralat lalai boleh mendedahkan jenis pelayan dan laluan. Sediakan halaman ralat tersuai yang ringkas tanpa butiran teknikal:

```
ErrorDocument 403 /error.html
ErrorDocument 404 /error.html
ErrorDocument 500 /error.html
```

> **NOTA — Obscurity ialah lapisan, bukan perisai**
> Menyembunyikan versi menghalang imbasan automatik yang menyasarkan versi tertentu, tetapi penyerang berazam masih boleh mengecam Joomla melalui corak URL dan aset. Jadi sentiasa gandingkan langkah ini dengan kemas kini segera (Bab 9) dan WAF (Bab 8) — jangan bergantung pada obscurity semata-mata.

BAB 5

PHP-FPM

Konfigurasi PHP yang selamat untuk Joomla

PHP-FPM (FastCGI Process Manager) menjalankan kod PHP secara berasingan daripada Apache, memberi prestasi dan pengasingan yang lebih baik. Bab ini mengetatkan konfigurasi PHP untuk mengurangkan kesan jika kod berniat jahat berjaya dilaksanakan.

5.1 Pemasangan

Pasang PHP-FPM dan sambungan yang diperlukan Joomla. Sesuaikan nombor versi mengikut pelayan anda:

```
$ sudo apt install -y php-fpm php-mysql php-xml php-gd \
    php-curl php-mbstring php-zip php-intl
$ sudo a2enconf php8.5-fpm      # atau php8.4-fpm
$ sudo a2enmod proxy_fcgi setenvif
$ sudo systemctl reload apache2
```

5.2 Ketatkan php.ini

Sunting fail konfigurasi PHP-FPM (bukan versi CLI). Lokasi: `/etc/php/8.5/fpm/php.ini`.

```
# Jangan dedahkan versi PHP dalam header
expose_php = Off
# Jangan papar ralat kepada pelawat (produksi)
display_errors = Off
log_errors = On
# Hadkan muat naik & memori
upload_max_filesize = 32M
post_max_size = 32M
memory_limit = 256M
max_execution_time = 60
# Kuki sesi lebih selamat
session.cookie_httponly = 1
session.cookie_secure = 1
session.use_strict_mode = 1
```

5.3 Matikan fungsi berbahaya

Banyak web-shell bergantung pada fungsi pelaksanaan sistem. Matikan yang tidak diperlukan Joomla:

```
disable_functions = exec,passthru,shell_exec,system,proc_open,popen,\
    curl_multi_exec,parse_ini_file,show_source
```

AMARAN — Uji selepas menyekat fungsi

Sesetengah sambungan Joomla mungkin memerlukan fungsi tertentu. Jika sesuatu rosak selepas perubahan ini, semak log ralat PHP dan buang fungsi berkenaan daripada senarai — jangan matikan keseluruhan senarai secara membuta.

5.4 open_basedir & kolam khusus

Hadkan PHP supaya hanya boleh mengakses direktori laman, menghalang traversal ke bahagian lain sistem fail. Sunting kolam FPM: `/etc/php/8.5/fpm/pool.d/www.conf`:

```
# Kurung PHP dalam direktori laman + tmp
php_admin_value[open_basedir] = /var/www/joomla:/tmp:/var/lib/php
# Log ralat khusus
php_admin_value[error_log] = /var/log/php/joomla-error.log
php_admin_flag[log_errors] = on
```

```
$ sudo mkdir -p /var/log/php && sudo chown www-data:www-data /var/log/php
$ sudo systemctl restart php8.5-fpm
```

BAB 6

MySQL

Pangkalan data selamat dengan keistimewaan minimum

Pangkalan data menyimpan segala kandungan dan kredential Joomla. Prinsip utama: MySQL hanya mendengar secara tempatan, akaun root dilindungi, dan Joomla diberi pengguna khusus dengan keistimewaan paling minimum.

6.1 Pemasangan

```
$ sudo apt install -y mysql-server
```

Pemasangan lalai MySQL selamat untuk permulaan, tetapi ia meninggalkan beberapa tetapan longgar — akaun tanpa nama, pangkalan data ujian, dan akses root yang belum dikukuhkan. Kita betulkan semua ini dengan skrip terbina yang datang bersama MySQL.

6.2 Skrip mysql_secure_installation — kenapa ia penting

MySQL menyertakan skrip pengukuhan asas bernama `mysql_secure_installation`. Ia ialah **langkah keselamatan pertama dan paling penting** selepas pemasangan — ia menutup lubang yang wujud secara lalai yang kerap dieksploit oleh imbasan automatik. Jalankan sekali sahaja, sebaik selepas pemasangan:

```
$ sudo mysql_secure_installation
```

Skrip akan bertanya beberapa soalan mengikut urutan. Berikut ialah setiap soalan, cara menjawab, dan **sebab** ia penting:

VALIDATE PASSWORD component	Pilih Yes untuk menguatkuasakan kata laluan kukuh. Ini memastikan semua kata laluan MySQL memenuhi standard minimum panjang dan kerumitan — pertahanan asas terhadap brute-force.
Set root password	Tetapkan kata laluan root yang kukuh dan unik. Akaun root MySQL mempunyai kawalan penuh ke atas semua pangkalan data; jika ia lemah atau kosong, keseluruhan pelayan terdedah.
Remove anonymous users	Jawab Yes. Akaun tanpa nama membenarkan sesiapa sahaja menyambung ke MySQL tanpa akaun sah — pintu belakang yang mesti ditutup.
Disallow root login remotely	Jawab Yes. Ini menghalang log masuk root dari rangkaian; root hanya boleh diakses dari pelayan itu sendiri (localhost). Menghapuskan sasaran brute-force jauh yang besar.
Remove test database	Jawab Yes. Pangkalan data 'test' lalai boleh diakses oleh mana-mana pengguna dan tidak diperlukan dalam produksi.

Reload privilege tables	Jawab Yes. Menggunakan semua perubahan di atas dengan serta-merta tanpa perlu memulakan semula MySQL.

AMARAN — Jawab Yes untuk semua (dalam kebanyakan kes)
Untuk pelayan produksi Joomla, jawapan selamat ialah **Yes** untuk setiap soalan. Satu-satunya pertimbangan: jika anda menggunakan alat yang memerlukan sambungan root jauh (jarang, dan tidak disyorkan), anda mungkin perlu menyesuaikan 'disallow root remotely' — tetapi bagi Joomla, root jauh tidak diperlukan langsung.

NOTA — Skrip ini melengkapi, bukan menggantikan
`mysql_secure_installation` menutup lubang lalai, tetapi ia tidak mencipta pengguna Joomla atau mengehadkan bind-address. Langkah-langkah itu (6.3 dan 6.4) sama penting — skrip ini hanya titik permulaan.

6.3 Pastikan hanya mendengar tempatan

Sahkan MySQL terikat pada localhost sahaja supaya tidak terdedah ke rangkaian. Semak `/etc/mysql/mysql.conf.d/mysqld.cnf`:

```
[mysqld]
bind-address = 127.0.0.1
mysqlx-bind-address = 127.0.0.1
```

```
$ sudo ss -tlnp | grep 3306   # sahkan hanya 127.0.0.1:3306
```

AMARAN — Jangan dedah port 3306
Jika output menunjukkan `0.0.0.0:3306`, pangkalan data anda terdedah ke internet. Betulkan bind-address dan pastikan UFW tidak membenarkan port 3306.

6.4 Cipta pengguna & pangkalan data Joomla

Jangan gunakan akaun root untuk Joomla. Cipta pengguna khusus dengan akses terhad kepada satu pangkalan data sahaja:

```
$ sudo mysql
mysql> CREATE DATABASE joomla_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
mysql> CREATE USER 'joomla_user'@'localhost' IDENTIFIED BY 'KataLaluanKukuh!2026';
mysql> GRANT SELECT, INSERT, UPDATE, DELETE, CREATE, DROP, INDEX, ALTER,
       CREATE TEMPORARY TABLES, LOCK TABLES ON joomla_db.* TO 'joomla_user'@'localhost';
mysql> FLUSH PRIVILEGES;
mysql> EXIT;
```

NOTA — Keistimewaan minimum
Perhatikan pengguna Joomla hanya diberi hak pada `joomla_db.*`, bukan seluruh pelayan. Ia tidak mempunyai hak pentadbiran seperti GRANT atau SUPER. Jika Joomla dikompromi, kerosakan terkurung pada satu pangkalan data sahaja.

BAB 7

Let's Encrypt (TLS)

Sijil percuma, HTTPS wajib, auto-renew

Trafik yang tidak disulitkan boleh dipintas — termasuk kata laluan pentadbir Joomla. Let’s Encrypt menyediakan sijil TLS percuma yang diperbaharui secara automatik melalui alat Certbot.

7.1 Pasang Certbot & dapatkan sijil

```
$ sudo apt install -y certbot python3-certbot-apache
$ sudo certbot --apache -d contoh.my -d www.contoh.my
```

Certbot akan mendapatkan sijil, mengubah suai VirtualHost anda untuk HTTPS, dan menyediakan pengalihan HTTP ke HTTPS secara automatik. Pilih pilihan 'redirect' apabila ditanya.

7.2 Konfigurasi SSL kukuh + HSTS

Setelah TLS aktif, tambah header HSTS untuk memaksa pelayar sentiasa menggunakan HTTPS. Masukkan dalam VirtualHost port 443:

```
Header always set Strict-Transport-Security "max-age=63072000; includeSubDomains; preload"
```

AMARAN — HSTS bersifat kekal

Setelah HSTS dihantar dengan tempoh panjang, pelayar akan enggan menyambung melalui HTTP untuk domain itu sehingga tempoh tamat. Pastikan HTTPS benar-benar berfungsi sebelum menetapkan max-age yang panjang.

7.3 Sahkan auto-renew

Certbot memasang pemasa systemd yang memperbaharui sijil secara automatik sebelum tamat tempoh. Uji prosesnya tanpa benar-benar memperbaharui:

```
$ sudo certbot renew --dry-run
$ sudo systemctl list-timers | grep certbot
```

BAB 8

ModSecurity + OWASP CRS

Firewall aplikasi web menapis serangan

ModSecurity ialah firewall aplikasi web (WAF) yang menapis permintaan HTTP sebelum ia sampai ke Joomla. Digandingkan dengan OWASP Core Rule Set (CRS), ia menghalang corak serangan biasa: SQL injection, XSS, command injection, dan traversal fail.

8.1 Pasang ModSecurity

```
$ sudo apt install -y libapache2-mod-security2
$ sudo cp /etc/modsecurity/modsecurity.conf-recommended \
     /etc/modsecurity/modsecurity.conf
$ sudo a2enmod security2
```

8.2 Aktifkan mod penguatkuasaan

Secara lalai ModSecurity hanya *mengesan* (DetectionOnly). Setelah anda yakin, tukar kepada mod menyekat. Sunting `/etc/modsecurity/modsecurity.conf`:

```
# Daripada: SecRuleEngine DetectionOnly
SecRuleEngine On
```

> **NOTA — Mula dengan DetectionOnly**
> Biarkan `DetectionOnly` selama beberapa hari dan pantau `/var/log/apache2/modsec_audit.log`. Ini membolehkan anda mengenal pasti positif palsu sebelum mengaktifkan penyekatan sebenar.

8.3 Pasang OWASP CRS 4

Ubuntu menyediakan pakej CRS, tetapi versi terkini (CRS 4.x) sebaiknya diambil terus daripada repositori rasmi untuk peraturan paling kemas kini:

```
$ sudo apt install -y modsecurity-crs
# atau versi terkini dari coreruleset.org (CRS 4.26 / LTS 4.25)
$ sudo git clone https://github.com/coreruleset/coreruleset \
     /etc/apache2/modsecurity-crs
$ sudo cp /etc/apache2/modsecurity-crs/crs-setup.conf.example \
     /etc/apache2/modsecurity-crs/crs-setup.conf
```

Rujuk CRS dalam konfigurasi ModSecurity Apache (`/etc/apache2/mods-enabled/security2.conf`):

```
<IfModule security2_module>
    SecDataDir /var/cache/modsecurity
    IncludeOptional /etc/modsecurity/*.conf
    IncludeOptional /etc/apache2/modsecurity-crs/crs-setup.conf
    IncludeOptional /etc/apache2/modsecurity-crs/rules/*.conf
</IfModule>
```

```
$ sudo apache2ctl configtest
$ sudo systemctl reload apache2
```

8.4 Paranoia level & pengecualian Joomla

CRS menggunakan 'paranoia level' (PL) 1 hingga 4 — makin tinggi makin ketat tetapi makin banyak positif palsu. Mula pada PL1. Editor Joomla dan sesetengah sambungan mungkin mencetuskan peraturan; tambah pengecualian sasaran, bukan mematikan keseluruhan peraturan.

```
# contoh pengecualian dalam crs-setup.conf atau fail before-crs
# matikan satu peraturan yang mencetuskan positif palsu pada admin
SecRuleRemoveById 941100
```

AMARAN — Jangan matikan CRS sepenuhnya

Jika berlaku positif palsu, kenal pasti ID peraturan dalam log audit dan buat pengecualian khusus. Mematikan keseluruhan CRS menghapuskan perlindungan WAF anda.

BAB 9

Pengukuhan Joomla

Selamatkan CMS itu sendiri

Dengan pelayan yang kukuh, kini kita selamatkan Joomla. Kebanyakan pencerobohan berpunca daripada sambungan pihak ketiga dan konfigurasi yang lemah, jadi bab ini memberi tumpuan pada kedua-duanya.

9.1 Keizinan fail & pemilikan

Fail dimiliki oleh pengguna web, dengan direktori 755 dan fail 644. Fail configuration.php dikunci lebih ketat kerana ia mengandungi kredential pangkalan data:

```
$ sudo chown -R www-data:www-data /var/www/joomla
$ sudo find /var/www/joomla -type d -exec chmod 755 {} \;
$ sudo find /var/www/joomla -type f -exec chmod 644 {} \;
$ sudo chmod 400 /var/www/joomla/configuration.php
```

9.2 Tetapan configuration.php

Matikan paparan ralat dan aktifkan penulisan semula URL yang selamat. Dalam `configuration.php`:

```
public $error_reporting = 'none';
public $sef = '1';
public $sef_rewrite = '1';
public $force_ssl = '2';   // paksa HTTPS untuk seluruh laman
public $tmp_path = '/var/www/joomla/tmp';
public $log_path = '/var/www/joomla/administrator/logs';
```

9.3 Aktifkan .htaccess Joomla

Joomla menyertakan fail perlindungan bernama `htaccess.txt`. Namakan semula untuk mengaktifkannya — ia menyekat banyak corak eksploit pada peringkat Apache:

```
$ cd /var/www/joomla
$ sudo -u www-data mv htaccess.txt .htaccess
```

9.4 Akaun & Super User

- Tukar nama pengguna Super User lalai (elakkan 'admin').
- Guna kata laluan unik ≥ 12 aksara untuk setiap akaun pentadbir.
- Aktifkan pengesahan dua faktor (2FA) terbina Joomla untuk semua akaun keistimewaan tinggi.
- Hadkan bilangan akaun Super User seminimum mungkin.

9.5 Sambungan (extensions)

- Pasang sambungan hanya daripada Joomla Extensions Directory (JED) atau vendor rasmi.
- Semak Vulnerable Extensions List (vel.joomla.org) sebelum memasang.
- Nyahpasang sepenuhnya sambungan yang tidak digunakan — bukan sekadar nyahaktif.
- Kemas kini teras Joomla dan semua sambungan sebaik sahaja tampung dikeluarkan.

BAB 10

Fail2ban

Sekat penyerang berulang secara automatik

Fail2ban memantau fail log dan mengharamkan (ban) alamat IP yang menunjukkan tingkah laku berniat jahat — seperti percubaan log masuk gagal berulang. Ia melengkapkan firewall dengan tindak balas dinamik.

10.1 Pemasangan

```
$ sudo apt install -y fail2ban
$ sudo cp /etc/fail2ban/jail.conf /etc/fail2ban/jail.local
```

10.2 Konfigurasi jail

Sunting `/etc/fail2ban/jail.local`. Tetapkan tempoh ban dan bilangan cubaan:

```
[DEFAULT]
bantime  = 1h
findtime = 10m
maxretry = 5

[sshd]
enabled = true
port = 22

[apache-auth]
enabled = true

[apache-badbots]
enabled = true
```

```
$ sudo systemctl enable --now fail2ban
$ sudo fail2ban-client status sshd
```

10.3 Jail khusus login Joomla (pilihan)

Anda boleh mencipta penapis tersuai yang memadankan percubaan log masuk gagal dalam log Joomla, kemudian menyambungkannya ke jail. Ini memerlukan log masuk Joomla direkodkan — aktifkan pengelogan dalam tetapan Global Configuration Joomla, kemudian tuding penapis Fail2ban ke fail log tersebut.

BAB 11

Backup & pemulihan

Jaring keselamatan terakhir anda

Tiada sistem yang kalis serangan sepenuhnya. Sandaran yang kerap dan telah diuji ialah perbezaan antara gangguan kecil dan bencana kekal. Sandarkan kedua-dua fail dan pangkalan data.

11.1 Skrip sandaran

Skrip mudah yang menyandarkan pangkalan data dan fail laman ke direktori bertarikh:

```
#!/usr/bin/env bash
set -euo pipefail
STAMP=$(date +%Y%m%d-%H%M%S)
DEST=/var/backups/joomla/$STAMP
mkdir -p "$DEST"
# Pangkalan data
mysqldump --single-transaction joomla_db | gzip > "$DEST/db.sql.gz"
# Fail laman
tar czf "$DEST/files.tar.gz" -C /var/www joomla
# Simpan 14 sandaran terakhir sahaja
ls -1dt /var/backups/joomla/*/ | tail -n +15 | xargs -r rm -rf
```

```
$ sudo install -m 700 backup-joomla.sh /usr/local/sbin/
$ echo '0 3 * * * root /usr/local/sbin/backup-joomla.sh' | \
    sudo tee /etc/cron.d/joomla-backup
```

11.2 Uji pemulihan

AMARAN — Sandaran tidak diuji = tiada sandaran

Sandaran hanya berguna jika ia boleh dipulihkan. Secara berkala, pulihkan sandaran ke pelayan pementasan dan sahkan laman berfungsi. Simpan juga salinan luar tapak (off-site).

```
# pulihkan pangkalan data
$ gunzip < db.sql.gz | mysql joomla_db
# pulihkan fail
$ tar xzf files.tar.gz -C /var/www
```

BAB 12

Pemantauan & penyelenggaraan

Kekal waspada selepas ke produksi

Keselamatan ialah proses berterusan, bukan tugas sekali selesai. Pemantauan membolehkan anda mengesan dan bertindak balas terhadap serangan sebelum ia menjadi pelanggaran penuh.

12.1 Log yang perlu dipantau

Apache akses/ralat	/var/log/apache2/*.log
Audit ModSecurity	/var/log/apache2/modsec_audit.log
PHP-FPM	/var/log/php/joomla-error.log
Fail2ban	/var/log/fail2ban.log
Log auth sistem	/var/log/auth.log

12.2 Pantau integriti fail

Serangan hampir selalu meninggalkan kesan — fail baharu atau yang diubah suai. Alat seperti AIDE mengesan perubahan pada fail teras:

```
$ sudo apt install -y aide
$ sudo aideinit
$ sudo aide --check   # bandingkan keadaan semasa dengan baseline
```

12.3 Rutin penyelenggaraan

- Kemas kini teras Joomla, sambungan, dan pakej sistem sebaik sahaja tampung keluar.
- Semak log secara berkala untuk corak serangan (XSS, SQLi, brute-force).
- Jalankan imbasan keselamatan luaran (contoh SecScan) secara berjadual dan selesaikan isu kritikal.
- Sahkan sandaran berjaya dan boleh dipulihkan setiap bulan.
- Semak semula akaun pengguna — buang yang tidak lagi diperlukan.

BAB 13

CDN & Cloudflare

Pertahanan pinggir sebelum trafik sampai ke pelayan

Sehingga kini semua pertahanan berada pada pelayan itu sendiri. Rangkaian penyampaian kandungan (CDN) seperti Cloudflare menambah lapisan pertama di **pinggir rangkaian** — trafik ditapis dan cache dihidangkan sebelum sampai ke pelayan anda. Ini menyerap serangan DDoS, menyembunyikan IP asal, dan mengurangkan beban.

13.1 Apa yang Cloudflare tambah

- **WAF pinggir** — menapis corak serangan sebelum ia menyentuh Apache/ModSecurity.
- **Perlindungan DDoS** — menyerap banjiran trafik pada rangkaian Cloudflare, bukan pelayan anda.
- **Sembunyi IP asal** — penyerang melihat IP Cloudflare, bukan pelayan sebenar.
- **Cache & TLS** — kandungan statik dihidangkan pantas; sijil pinggir automatik.
- **Rate limiting** — hadkan permintaan ke `/administrator` Joomla.

13.2 Persediaan asas

1. Daftar domain di Cloudflare dan tukar nameserver domain anda kepada nameserver Cloudflare.
2. Pastikan rekod DNS untuk laman anda berstatus ‘Proxied’ (awan oren) supaya trafik melalui Cloudflare.
3. Tetapkan mod SSL/TLS kepada **Full (strict)** supaya sambungan Cloudflare ke pelayan anda juga disulitkan dan sijilnya disahkan.
4. Aktifkan ‘Always Use HTTPS’ dan ‘Automatic HTTPS Rewrites’.

> **AMARAN — Full (strict) memerlukan sijil sah**
> Mod **Full (strict)** menuntut pelayan asal anda mempunyai sijil TLS yang sah — yang telah kita sediakan dengan Let’s Encrypt di Bab 7. Jangan guna mod ‘Flexible’ kerana ia meninggalkan sambungan Cloudflare-ke-pelayan tidak disulitkan.

13.3 Kunci pelayan supaya hanya terima trafik Cloudflare

Selepas Cloudflare aktif, penyerang masih boleh memintas Cloudflare jika mereka tahu IP asal anda dan menyambung terus. Halang ini dengan hanya membenarkan julat IP Cloudflare pada firewall untuk port 80/443:

```
# Contoh — benarkan hanya julat Cloudflare, kemudian tolak selebihnya.
# Senarai penuh: https://www.cloudflare.com/ips/
$ for ip in $(curl -s https://www.cloudflare.com/ips-v4); do
    sudo ufw allow from $ip to any port 443 proto tcp
  done
$ sudo ufw delete allow 443/tcp   # buang kebenaran umum 443
```

> **NOTA — Kembalikan IP sebenar pelawat**
> Apabila trafik melalui Cloudflare, Apache melihat IP Cloudflare, bukan pelawat. Pasang modul `libapache2-mod-remoteip` dan konfigur `CF-Connecting-IP` supaya log dan Fail2ban mengharamkan IP pelawat sebenar, bukan IP Cloudflare.

13.4 Peraturan keselamatan berguna

- Cipta peraturan WAF yang menghadkan kadar (rate limit) ke `/administrator` — contoh maksimum 5 percubaan seminit setiap IP.
- Sekat negara atau ASN yang tidak sepatutnya mengakses panel pentadbir jika audiens anda tempatan.
- Aktifkan ‘Bot Fight Mode’ untuk menapis bot automatik yang menyerang borang log masuk.
- Guna ‘Managed Challenge’ untuk permintaan mencurigakan ke laluan sensitif.

BAB 14

Pemulihan selepas diserang

Bertindak tenang, sistematik, dan menyeluruh

Walaupun dengan pertahanan terbaik, pelanggaran masih boleh berlaku. Cara anda bertindak balas menentukan sama ada ia gangguan kecil atau bencana. Bab ini memberi pelan tindakan berperingkat. Prinsip utama: **jangan panik, jangan padam bukti dahulu, dan jangan sekadar 'bersihkan' tanpa mencari punca**.

14.1 Kuarantin — hadkan kerosakan

Langkah pertama ialah menghentikan pendarahan tanpa memusnahkan bukti forensik.

1. Ambil laman ke luar talian atau aktifkan 'Under Attack Mode' Cloudflare untuk menyekat trafik.
2. **Jangan** memadam fail atau memformat dahulu — anda memerlukannya untuk memahami cara serangan berlaku.
3. Ambil snapshot/imej penuh pelayan (fail + pangkalan data) untuk analisis forensik.
4. Tukar SEMUA kata laluan daripada mesin yang bersih: Super User Joomla, pangkalan data, SSH, cPanel/hos.

> **AMARAN — Andaikan kredential telah bocor**
> Jika penyerang mendapat akses, anggap semua kata laluan dan kunci API yang tersimpan pada pelayan telah terdedah. Jana semula kesemuanya, termasuk 'Secret' dalam configuration.php Joomla dan kunci API mana-mana sambungan.

14.2 Siasat — cari punca & skop

Gunakan log yang telah anda kumpulkan (Bab 12) untuk memahami apa yang berlaku:

- Semak `access.log` untuk permintaan POST mencurigakan ke fail PHP yang tidak dikenali.
- Cari fail yang diubah suai baru-baru ini: `find /var/www/joomla -type f -mtime -7 -name '*.php'`.
- Jalankan pemeriksaan integriti AIDE untuk menyenaraikan fail yang berubah daripada baseline.
- Cari web-shell: corak seperti `eval(`, `base64_decode(`, `gzinflate(` dalam fail PHP yang baru muncul.
- Semak akaun pengguna Joomla dan pangkalan data untuk pengguna 'hantu' yang ditambah penyerang.

```
# Cari kemungkinan web-shell dalam direktori laman
$ sudo grep -rEl "eval\(|base64_decode\(|gzinflate\(|str_rot13\(" \
    /var/www/joomla --include=*.php
```

14.3 Pulih — bina semula dengan bersih

Cara paling selamat untuk memulihkan laman yang dikompromi ialah membina semula daripada sumber yang dipercayai, bukan sekadar memadam fail yang disyaki.

1. Pulihkan daripada sandaran yang diketahui bersih (**sebelum** tarikh pencerobohan).
2. Jika tiada sandaran bersih: pasang Joomla teras yang baharu, pasang semula sambungan daripada sumber rasmi, kemudian pindahkan hanya kandungan (artikel, media) yang telah disemak.
3. Kemas kini teras Joomla dan setiap sambungan ke versi terkini — punca kemasukan selalunya kelemahan sambungan yang belum ditampung.
4. Jana semula semua kata laluan dan rahsia sekali lagi selepas pemulihan.
5. Imbas semula fail sebelum kembali ke talian, kemudian pantau rapat beberapa hari.

> **NOTA — Selepas pulih: laporkan & kukuhkan**
> Jika ini kelemahan sambungan baharu, laporkan kepada Joomla Security Task Force (bukan forum awam). Kemudian semak semula setiap bab manual ini — pelanggaran biasanya menunjukkan satu lapisan yang terlepas. Perkukuh lapisan itu supaya serangan yang sama tidak berulang.

14.4 Senarai tindakan pantas insiden

Kuarantin	Ambil luar talian / Under Attack Mode; ambil snapshot forensik
Kredential	Tukar semua kata laluan & kunci dari mesin bersih
Siasat	Log, fail berubah, web-shell, pengguna hantu
Pulih	Bina semula dari sumber bersih; kemas kini semua
Kukuh	Tampung lapisan yang terlepas; pantau; laporkan

BAB 15

Nginx (alternatif Apache)

Jika anda menggunakan Nginx dan bukan Apache 2

Bab-bab terdahulu menganggap Apache 2.4. Jika pelayan anda menggunakan **Nginx**, prinsip keselamatan kekal sama — hanya sintaks konfigurasi berbeza. Bab ini memberikan padanan Nginx untuk tetapan penting: blok pelayan Joomla, penulisan semula URL SEF, header keselamatan, sekatan fail sensitif, dan TLS. Anda tidak perlu memasang Apache jika menggunakan Nginx.

> **NOTA — Nginx tiada .htaccess**
> Berbeza dengan Apache, Nginx tidak membaca fail `.htaccess`. Semua peraturan mesti diletakkan terus dalam blok pelayan (server block). Ini bermakna fail `htaccess.txt` Joomla tidak digunakan; sebaliknya kita tulis peraturan setara dalam konfigurasi Nginx.

15.1 Pemasangan Nginx + PHP-FPM

```
$ sudo apt install -y nginx php-fpm php-mysql php-xml php-gd \
    php-curl php-mbstring php-zip php-intl
$ sudo systemctl enable --now nginx php8.5-fpm
```

Nginx tidak menjalankan PHP secara langsung — ia menghantar permintaan PHP kepada PHP-FPM melalui soket FastCGI. Semua pengukuhan PHP-FPM dalam Bab 5 (disable_functions, open_basedir, had) terpakai sepenuhnya.

15.2 Blok pelayan untuk Joomla

Cipta konfigurasi tapak di `/etc/nginx/sites-available/joomla`. `try_files` ialah kunci kepada URL SEF Joomla yang selamat:

```
server {
    listen 80;
    server_name contoh.my www.contoh.my;
    root /var/www/joomla;
    index index.php;

    # URL SEF Joomla — hantar ke index.php jika fail tiada
    location / {
        try_files $uri $uri/ /index.php?$args;
    }

    # Serah PHP kepada PHP-FPM
    location ~ \.php$ {
        include snippets/fastcgi-php.conf;
        fastcgi_pass unix:/run/php/php8.5-fpm.sock;
    }
}
```

15.3 Sekat fail & direktori sensitif

Tambah blok ini dalam `server { }` untuk menghalang akses ke fail konfigurasi, fail tersembunyi, dan direktori sistem Joomla yang tidak sepatutnya dihidangkan terus:

```
# Halang configuration.php & fail sensitif
location ~* /(configuration\.php|\.env|\.git) {
    deny all;
}
# Halang fail tersembunyi & backup
location ~ /\.(?!well-known) { deny all; }
location ~* \.(bak|save|old|sql|log)$ { deny all; }
# Halang PHP dalam direktori muat naik/tmp
location ~* /(images|tmp|cache)/.*\.php$ { deny all; }
```

> **AMARAN — Jangan benarkan PHP dalam folder muat naik**
>
> Peraturan terakhir di atas penting: ia menghalang pelaksanaan skrip PHP yang mungkin dimuat naik ke `images/` atau `tmp/` — teknik biasa untuk menanam web-shell.

15.4 Header keselamatan

Padanan Nginx untuk header keselamatan (setara Bab 4). Letak dalam blok server:

```
add_header X-Content-Type-Options "nosniff" always;
add_header X-Frame-Options "SAMEORIGIN" always;
add_header Referrer-Policy "strict-origin-when-cross-origin" always;
# Sembunyikan versi Nginx
server_tokens off;
```

```
$ sudo ln -s /etc/nginx/sites-available/joomla /etc/nginx/sites-enabled/
$ sudo nginx -t          # uji sintaks
$ sudo systemctl reload nginx
```

15.5 TLS dengan Certbot untuk Nginx

Sama seperti Bab 7, tetapi gunakan pemalam Nginx. Certbot akan mengubah blok pelayan untuk HTTPS dan menambah pengalihan automatik:

```
$ sudo apt install -y certbot python3-certbot-nginx
$ sudo certbot --nginx -d contoh.my -d www.contoh.my
```

Tambah HSTS selepas HTTPS berfungsi: `add_header Strict-Transport-Security "max-age=63072000; includeSubDomains; preload" always;` dalam blok pelayan port 443.

> **NOTA — ModSecurity untuk Nginx**
> WAF ModSecurity juga tersedia untuk Nginx melalui modul `ModSecurity-nginx` dengan OWASP CRS yang sama (Bab 8). Pemasangan berbeza sedikit — ia dimuatkan sebagai modul dinamik Nginx — tetapi peraturan CRS dan konsep paranoia level adalah sama.

BAB 16

Jailkit dengan Virtualmin

Kurung pengguna SSH dalam chroot jail

Jika anda menguruskan pelayan dengan **Virtualmin** dan mempunyai beberapa domain atau pelanggan, setiap pemilik domain yang mempunyai akses SSH boleh — secara lalai — melihat fail di luar direktori mereka sendiri. **Jailkit** menyelesaikan ini dengan mengurung (chroot jail) setiap pengguna supaya mereka hanya nampak direktori laman mereka, bukan keseluruhan sistem fail pelayan.

Ini penting untuk hos berbilang penyewa: jika satu akaun dikompromi, penyerang terperangkap dalam jail domain itu dan tidak boleh membaca konfigurasi atau data domain lain.

16.1 Jailkit sudah ada dalam Virtualmin

Sejak Virtualmin 6, Jailkit dipasang secara lalai. Penting: gunakan pakej Jailkit yang dibina oleh pasukan Virtualmin — ia dibina dengan *capabilities* Linux dan bukan binari suid, menjadikannya lebih selamat daripada pakej Jailkit generik.

```
# Sahkan Jailkit dipasang (dari repo Virtualmin)
$ dpkg -l | grep jailkit
$ which jk_init jk_chrootsh
```

AMARAN — Jangan pasang Jailkit generik

Jangan pasang Jailkit daripada sumber lain atau kompil sendiri untuk kegunaan Virtualmin — versi Virtualmin diubah suai khas. Pakej yang tidak serasi boleh menyebabkan jail gagal berfungsi atau menimbulkan isu keselamatan.

16.2 Aktifkan jail untuk domain baharu

Untuk menjadikan semua pengguna domain baharu dikurung secara automatik, ubah suai templat pelayan:

1. Buka Virtualmin → System Settings → **Server Templates**.
2. Pilih templat yang berkenaan, kemudian bahagian **Administration user**.
3. Tetapkan '**Chroot jail new domain Unix users?**' kepada **Yes**.
4. Simpan. Domain baharu yang dicipta selepas ini akan mempunyai pengguna yang dikurung.

16.3 Kurung domain sedia ada

Untuk domain yang sudah wujud, aktifkan jail pada peringkat pemilik:

1. Buka Virtualmin dan pilih domain berkenaan.
2. Pergi ke **Administration Options** → **Edit Owner Limits**.
3. Dalam bahagian **Other restrictions**, tetapkan '**Chroot jail domain Unix user?**' kepada **Yes**.
4. Ini mengurung pemilik domain dan semua penggunanya.

> **NOTA — Modul Webmin pilihan**
> Untuk mengurus fail konfigurasi jail secara visual, anda boleh memasang modul `wbm-jailkit`. Virtualmin tidak memerlukannya untuk berfungsi, tetapi ia memudahkan penyesuaian binari yang tersedia dalam jail.

16.4 Uji jail berfungsi

Selepas mengaktifkan, sahkan pengguna benar-benar terkurung. Log masuk sebagai pengguna domain melalui SSH dan cuba lihat sistem fail:

```
$ ssh pengguna_domain@pelayan
# dalam jail — 'ls /' hanya menunjukkan direktori jail, bukan sistem penuh
$ ls /
$ pwd    # sepatutnya di dalam /home/domain/... sahaja
```

Jika pengguna boleh menavigasi ke `/etc` atau melihat fail domain lain, jail belum aktif dengan betul — semak semula tetapan templat dan had pemilik.

16.5 Had & pertimbangan

- Jail mengehadkan binari yang tersedia. Jika pengguna memerlukan alat tertentu (contoh git, composer), ia mesti ditambah ke dalam jail melalui konfigurasi Jailkit.
- Tugas cron untuk pengguna yang dikurung mungkin memerlukan penyesuaian kerana persekitaran jail berbeza.
- Jailkit menguatkan pengasingan tetapi bukan pengganti keizinan fail yang betul (Bab 9) — gunakan kedua-duanya.

BAB 17

.htaccess & robots.txt

Dua fail kecil yang sering disalahfaham

Dua fail teks ringkas di root laman anda memainkan peranan berbeza. **.htaccess** ialah kawalan keselamatan sebenar (pada Apache); **robots.txt** pula BUKAN kawalan keselamatan langsung, walaupun ramai menganggapnya begitu. Bab ini menjelaskan kedua-duanya.

17.1 .htaccess — kawalan keselamatan Apache

Kita telah mengaktifkan htaccess.txt Joomla di Bab 9. Fail ini menyekat banyak corak eksploit pada peringkat Apache. Di bawah ialah peraturan pengukuhan tambahan yang boleh anda masukkan. Ingat: .htaccess hanya berfungsi pada Apache dengan `AllowOverride All` (Bab 4); pengguna Nginx guna blok pelayan (Bab 15).

```
# Matikan penyenaraian direktori
Options -Indexes

# Lindungi configuration.php sepenuhnya
<Files "configuration.php">
    Require all denied
</Files>

# Sekat fail tersembunyi, backup & log
<FilesMatch "(^\.|\.(bak|save|old|sql|log|ini)$)">
    Require all denied
</FilesMatch>
```

Halang pelaksanaan PHP dalam folder yang sepatutnya hanya menyimpan media — teknik penting menentang web-shell yang dimuat naik:

```
# Letak fail .htaccess ini dalam images/ dan tmp/
<FilesMatch "\.(php|phtml|php[0-9]|phar)$">
    Require all denied
</FilesMatch>
```

AMARAN — Uji selepas menambah peraturan

Peraturan .htaccess yang salah boleh menyebabkan ralat 500 atau menghalang laman berfungsi. Tambah sedikit demi sedikit dan uji laman selepas setiap perubahan. Simpan salinan htaccess.txt asal sebagai rujukan.

17.2 robots.txt — untuk crawler, BUKAN keselamatan

Joomla disertakan dengan fail `robots.txt` lalai. Fungsinya ialah memberitahu enjin carian (Google, Bing) bahagian mana yang tidak perlu diindeks. Ia berkaitan SEO, bukan keselamatan.

AMARAN — robots.txt tidak melindungi apa-apa

Robot yang jujur (Googlebot) mematuhi robots.txt, tetapi penyerang dan bot berniat jahat **mengabaikannya**. Lebih buruk lagi: menulis `Disallow: /administrator/` sebenarnya **memberitahu penyerang** di mana panel pentadbir berada. robots.txt ialah papan tanda, bukan kunci pintu.

Amalan yang betul untuk robots.txt:

- Gunakannya untuk SEO — halang pengindeksan halaman pendua atau hasil carian dalaman.
- JANGAN bergantung padanya untuk menyembunyikan kawasan sensitif — lindungi kawasan itu dengan pengesahan, kebenaran fail, dan .htaccess sebaliknya.
- Elakkan menyenaraikan laluan rahsia secara eksplisit; ia mendedahkan struktur laman.
- robots.txt lalai Joomla sudah memadai untuk kebanyakan laman — anda jarang perlu mengubahnya untuk tujuan keselamatan.

Contoh robots.txt lalai Joomla (fokus SEO, bukan menyembunyi):

```
User-agent: *
Disallow: /cache/
Disallow: /cli/
Disallow: /components/
Disallow: /includes/
Disallow: /tmp/
```

NOTA — Perlindungan sebenar /administrator

Untuk benar-benar melindungi `/administrator`, gunakan: kata laluan kukuh + 2FA (Bab 9), had kadar Cloudflare/Fail2ban (Bab 10, 13), dan jika boleh, sekatan IP atau BasicAuth tambahan. Itulah kunci pintu — robots.txt hanya papan tanda.

BAB 18

Menggunakan SecScan

Menjalankan scan dan mentafsir keputusan

Selepas mengukuhkan pelayan dan Joomla, bagaimana anda tahu ia berkesan? SecScan mengimbas laman anda secara luaran (black-box) — sama seperti yang dilihat penyerang — dan memberi gred, skor, serta senarai penemuan yang boleh ditindaki. Bab ini menerangkan cara menjalankannya dan, lebih penting, cara mentafsir keputusannya.

18.1 Menjalankan scan

1. Buka `https://security.apasaja.my` dalam pelayar.
2. Masukkan domain anda (contoh `contoh.my`) dalam medan scan.
3. Klik ‘Mula scan’ dan tunggu beberapa saat — SecScan menjalankan 30+ semakan.
4. Semak keputusan mengikut kategori, dan muat turun laporan jika perlu.

> **NOTA — Imbasan luaran melengkapi audit dalaman**
> SecScan melihat laman dari luar (header, TLS, DNS, fail terdedah). Ia melengkapi — bukan menggantikan — audit sisi-pelayan seperti skrip audit Ubuntu dan pemeriksaan dalam manual ini. Gunakan kedua-duanya.

18.2 Memahami gred & skor

SecScan memberi **skor 0–100** dan **gred A+ hingga F**. Skor dikira sebagai purata berwajaran semua semakan — setiap penemuan menyumbang mengikut keterukannya:

Lulus (pass)	100 mata — semakan lulus sepenuhnya
Info	85 mata — nasihat, bukan kegagalan keselamatan
Amaran (warning)	35 mata — isu yang patut dibetulkan
Kritikal	0 mata — kelemahan serius; betulkan segera

Skor akhir dipetakan kepada gred:

A+	90–100 — pengukuhan sangat baik
A	80–89 — baik
B	70–79 — memuaskan, ada ruang penambahbaikan
C	60–69 — sederhana; beberapa isu perlu perhatian
D	45–59 — lemah; banyak isu
F	bawah 45 — kritikal; tindakan segera diperlukan

AMARAN — Penemuan kritikal menghadkan gred anda

Walaupun skor purata tinggi, penemuan kritikal akan menurunkan gred: **1 kritikal** menghalang gred A+ (jadi A), **3 atau lebih** mengehadkan gred kepada maksimum C, dan **5 atau lebih** mengehadkan kepada D. Ini disengajakan — satu lubang kritikal boleh menjejaskan keseluruhan laman, tidak kira berapa baik selebihnya.

18.3 Empat kategori keterukan

Setiap penemuan dilabel dengan salah satu daripada empat kategori. Utamakan mengikut urutan ini:

- **Kritikal** — kelemahan yang boleh dieksploit secara langsung (contoh: fail configuration.php terdedah, TLS rosak). Betulkan dahulu, segera.
- **Amaran** — konfigurasi lemah yang meningkatkan risiko (contoh: header keselamatan hilang, DMARC p=none). Betulkan seterusnya.
- **Info** — pemerhatian bermaklumat, bukan kegagalan (contoh: tiada IPv6, DNSSEC tidak dikesan). Semak, tetapi tidak mendesak.
- **Lulus** — semakan yang berjaya. Sahkan ia kekal lulus selepas sebarang perubahan.

18.4 Skor mengikut kategori

SecScan memecahkan skor mengikut kategori supaya anda tahu bahagian mana yang paling lemah. Setiap kategori berkait terus dengan bab dalam manual ini:

TLS/SSL	Sijil, protokol, cipher → Bab 7 (Let's Encrypt)
Headers	Header keselamatan HTTP → Bab 4 / Bab 15
Exposed Files	Fail sensitif terdedah → Bab 9, Bab 17
Technology / CVE	Versi & CVE Joomla/PHP → Bab 9, Bab 12
DNS	SPF, DMARC, DNSSEC → konfigurasi domain
Cookies	Atribut Secure/HttpOnly → Bab 5, Bab 9

18.5 Cadangan diutamakan

SecScan menyusun senarai ‘Utamakan pembetulan ini’ — penemuan kritikal dan amaran diisih mengikut keutamaan, setiap satu dengan cadangan tindakan dan rujukan. Ikut senarai ini dari atas ke bawah: menyelesaikan item teratas memberi kesan terbesar pada gred anda.

NOTA — Petunjuk 'Quick win'

Sesetengah cadangan ditanda ‘Quick win’ — pembetulan berkesan tinggi tetapi mudah dilaksanakan (contoh menambah satu header). Selesaikan ini dahulu untuk peningkatan pantas.

18.6 PCI DSS Lite

Jika laman anda mengendalikan pembayaran, tab ‘PCI DSS Lite’ memberikan pandangan pematuhan ringkas. Statusnya: **Good alignment** (skor ≥ 85), **Partial alignment** (≥ 65), atau **Action required**. Ini bukan audit PCI rasmi, tetapi menonjolkan kawalan asas yang penting untuk laman e-dagang.

18.7 Gelung penambahbaikan

Gunakan SecScan sebagai gelung berterusan, bukan pemeriksaan sekali:

1. Jalankan scan untuk mendapat garis dasar (baseline) gred anda.
2. Selesaikan semua penemuan Kritikal, kemudian Amaran, mengikut senarai keutamaan.
3. Rujuk bab manual yang berkaitan untuk setiap pembetulan.
4. Jalankan scan semula untuk mengesahkan gred bertambah baik.
5. Ulang secara berkala — konfigurasi berubah, CVE baharu muncul, dan sijil tamat tempoh.

BAB 19

Senarai semak akhir

Audit ringkas sebelum ke produksi

Cetak halaman ini dan tandakan setiap item. Ia meringkaskan tindakan paling kritikal daripada keseluruhan manual.

Sistem & rangkaian

- [ ] Sistem dikemas kini; unattended-upgrades aktif
- [ ] Pengguna sudo bukan-root digunakan; root tidak digunakan untuk kerja harian
- [ ] UFW aktif — hanya 22/80/443 dibenarkan, 3306 tidak terdedah
- [ ] SSH: kunci awam sahaja, PermitRootLogin no, PasswordAuthentication no

Pelayan web & PHP

- [ ] Apache: ServerTokens Prod, header keselamatan dipasang, -Indexes
- [ ] configuration.php disekat akses terus; fail sensitif diblok
- [ ] PHP: expose_php Off, display_errors Off, disable_functions ditetapkan
- [ ] open_basedir mengurung PHP dalam direktori laman

Pangkalan data & TLS

- [ ] mysql_secure_installation dijalankan; MySQL mendengar 127.0.0.1 sahaja
- [ ] Pengguna Joomla khusus dengan keistimewaan minimum (bukan root)
- [ ] Sijil Let’s Encrypt aktif; HTTPS dipaksa; HSTS dihantar
- [ ] Auto-renew certbot disahkan (--dry-run lulus)

WAF, Joomla & pemantauan

- [ ] ModSecurity On dengan OWASP CRS 4; positif palsu ditangani secara sasaran
- [ ] Keizinan fail Joomla betul (755/644, configuration.php 400)
- [ ] htaccess.txt dinamakan semula; 2FA aktif; nama admin lalai ditukar
- [ ] Sambungan disemak terhadap vel.joomla.org; yang tidak guna dinyahpasang
- [ ] Fail2ban aktif; sandaran berjadual & diuji; log dipantau
- [ ] Imbasan keselamatan luaran dijalankan dan isu kritikal diselesaikan

Pinggir & kesediaan insiden

- [ ] Cloudflare aktif (Proxied); SSL/TLS Full (strict); IP asal disembunyikan
- [ ] Firewall hanya terima 80/443 daripada julat IP Cloudflare
- [ ] mod_remoteip dikonfigur supaya IP pelawat sebenar direkodkan
- [ ] Rate limit ke /administrator; Bot Fight Mode aktif
- [ ] Pelan tindak balas insiden difahami; sandaran bersih tersedia untuk pemulihan

Ikut persekitaran anda (pilihan)

- [ ] Jika Nginx: try_files SEF betul; PHP disekat dalam images/tmp; server_tokens off; TLS via certbot --nginx
- [ ] Jika Virtualmin: Jailkit aktif; pengguna domain dikurung dalam chroot jail dan telah diuji

---

Manual ini bahan rujukan teknikal. Keselamatan ialah pengurangan risiko, bukan penghapusan risiko. Sahkan setiap arahan pada persekitaran anda, kekalkan sandaran yang diuji, dan kekal berwaspada. Selamat · Terkawal · Dipercayai.